

Analytical and Numerical Simulation of an Electrical Field Mill

N. Plourde, *Student Member, IEEE*, H. Burch, *Member, IEEE*

Auburn Photonics and Sensing Lab (APSL) & Department of Electrical and Computer Engineering, Auburn University, Auburn, AL 36832 USA

Manuscript received October 7, 2026.

Abstract: [Code Github Link](#)

1. Introduction

The electric field mill is a widely used instrument for measuring static and slowly varying electric fields, with applications in thunderstorm and lightning warning, atmospheric electricity research, and electrostatic hazard monitoring. A direct measurement of a static field is difficult because the charge induced on a stationary electrode leaks away and the reading drifts. The field mill avoids this by placing a grounded, moving shutter above a set of sense plates, alternately exposing them to the ambient field and shielding them from it. The charge induced on the plates therefore varies periodically, which converts the static field into an alternating signal whose amplitude is proportional to the field strength.

Interpreting that signal requires knowing how much charge the ambient field induces on the sense plates and what voltage this produces across the measurement circuit. To first order, the induced charge follows from Gauss's law as the product of the permittivity of free space, the field strength, and the exposed plate area. In a real instrument, however, fringing at the shutter edges, the gap between the shutter and the plates, and the finite size of the electrodes all distort the field, so the idealized result is only an approximation.

This paper presents both an analytical calculation and a numerical simulation of a field mill. The analytical treatment derives the charge and voltage on the sense plates under idealized assumptions, giving closed-form expressions that show how the output scales with field strength and geometry. The numerical treatment solves for the electric potential on a two-dimensional cross-section of the instrument using Laplace's Update Equation, capturing the fringing and shielding effects the analytical model neglects. The resulting field distribution is presented as a heatmap of the two-dimensional electric field, and the charge and voltage obtained from the simulation are compared with the analytical predictions.

2. Analytical Calculation

The analysis begins with the definition of current as the time rate of change of charge on the sensing plate:

$$I = \frac{dQ_e}{dt}. \quad (1)$$

To find Q_e , we apply the boundary condition for the normal component of the electric flux density at the plate surface. Because the plate is assumed to be a perfect electrical conductor (PEC), the

field inside it vanishes, leaving

$$\hat{n} \cdot (\varepsilon_2 \mathbf{E}_2 - \varepsilon_1 \mathbf{E}_1)^0 = q_{es} \Rightarrow \hat{n} \cdot \varepsilon_2 \mathbf{E}_2 = q_{es}. \quad (2)$$

The total charge on the plate is found by integrating the surface charge density over the area. It is assumed that the plate is rectangular with width w and height h ,

$$Q_e = \iint_A q_{es} dA = \int_0^h \int_0^w \hat{n} \cdot \varepsilon_2 \mathbf{E}_2 dx dy. \quad (3)$$

Also, with the assumption that the field is uniform and normal to the plate. Region 2 is also assumed to free space so that $\varepsilon_2 = \varepsilon_0$ and $|\mathbf{E}_2| = E$, the integral reduces to

$$Q_e = \varepsilon_0 E wh = \varepsilon_0 EA. \quad (4)$$

Substituting this into (1) with E held constant gives

$$I = \varepsilon_0 E \frac{dA}{dt}, \quad (5)$$

where A is the area of the plate exposed to the field. As the plate moves with velocity v along its width, the exposed area changes at a rate of

$$\frac{dA}{dt} = h \frac{dw}{dt} = hv, \quad (6)$$

so the current from each plate is

$$I = \varepsilon_0 E hv. \quad (7)$$

Both plates feed a transimpedance amplifier, so the output voltage combines the two plate currents. Splitting the feedback resistor into two parallel halves lets each plate's contribution be calculated separately:

$$V_{\text{tot}} = IR = I_1 \frac{R}{2} - I_2 \frac{R}{2}. \quad (8)$$

Since the plates are identical, their currents are equal in magnitude and opposite in sign:

$$I_1 = -I_2. \quad (9)$$

The output voltage therefore becomes

$$V_{\text{tot}} = 2I_1 \frac{R}{2} = \varepsilon_0 E hv R. \quad (10)$$

With no fringe fields and PEC plates, the output voltage depends only on the plate velocity: for constant v , V_{tot} is constant, and any change in velocity produces a proportional change in the plate current and thus in the output voltage.

3. Numerical Simulation

The analytical model assumes each sense plate sees either the full uniform field or none of it. To capture fringing and partial shielding, the electrostatic potential is solved numerically on a two-dimensional cross-section of the mill. The mill is taken to be uniform along its depth h (into the page), so the potential depends only on the horizontal coordinate x and the vertical coordinate y .

3.1. Governing Equation

The region around the plates is charge-free, so the potential satisfies Laplace's equation,

$$\nabla^2 V = \frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = 0. \quad (11)$$

Replacing each derivative with a second-order central difference on a grid with spacings Δx and Δy , and solving for the centre node, gives the update equation

$$V_{i,j}^{(n+1)} = \frac{\Delta y^2 \left(V_{i+1,j}^{(n)} + V_{i-1,j}^{(n)} \right) + \Delta x^2 \left(V_{i,j+1}^{(n)} + V_{i,j-1}^{(n)} \right)}{2 (\Delta x^2 + \Delta y^2)}, \quad (12)$$

where i and j index x and y and n is the iteration number. This is Jacobi relaxation: every free node is repeatedly replaced by the weighted average of its four neighbours until the solution stops changing.

3.2. Domain, Geometry, and Boundary Conditions

The domain spans $1.5 \leq x \leq 8.5$ m and $2.5 \leq y \leq 8.5$ m on a 201×201 grid ($\Delta x = 3.5$ cm, $\Delta y = 3.0$ cm). Two sense plates, each 1.95 m wide and 0.2 m thick and separated by a 0.1 m gap, are centred at (5, 5) m. A grounded shutter 2 m wide and 0.2 m thick sits 1 m above them. The mill depth is $h = 4$ m.

The bottom edge is held at $V = 0$ and the top edge at $V = E_0(y_{\max} - y_{\min}) = 600$ V, which in the absence of the mill produces a uniform field $E_0 = 100$ V/m directed toward $-y$. The left and right edges use a zero-normal-derivative condition, $\partial V / \partial x = 0$, implemented with reflected ghost nodes. All three plates are held at $V = 0$: the shutter is grounded, and the sense plates sit at the virtual ground of the transimpedance amplifier. Nodes on these boundaries and inside the plates are excluded from the update.

3.3. Solution Procedure

The potential is initialised to the uniform-field solution $V = E_0(y - y_{\min})$ and iterated until the largest change at any node between successive iterations falls below 10^{-5} V. The field is then recovered from $\mathbf{E} = -\nabla V$ using central differences, and its magnitude is plotted as a heatmap.

3.4. Plate Charge

The charge on each plate follows from the discrete form of Gauss's law. The net flux of $\mathbf{E}$ leaving the cell around each node is summed over all nodes inside the plate P :

$$Q = \epsilon_0 h \sum_{(i,j) \in P} \left[(2V_{i,j} - V_{i+1,j} - V_{i-1,j}) \frac{\Delta y}{\Delta x} + (2V_{i,j} - V_{i,j+1} - V_{i,j-1}) \frac{\Delta x}{\Delta y} \right]. \quad (13)$$

Nodes in the interior of the plate contribute nothing, so the sum reduces to the flux through the plate surface. The two-dimensional sum gives charge per unit depth, and multiplying by h gives the total charge.

3.5. Shutter Sweep

The shutter is moved from fully covering Sense 1 to fully covering Sense 2 in steps of one grid spacing. At each position the potential is re-solved, using the previous solution as the starting guess to speed convergence, and the charge on each sense plate is recorded.

4. Results

Both models are evaluated for the same case: an ambient field of $E_0 = 100$ V/m, sense plates of width $w = 1.95$ m and depth $h = 4$ m ($A = 7.8$ m²), a feedback resistance of $R = 1$ k Ω , and a shutter velocity of $v = 1$ m/s. These values are arbitrary, since the expressions in Section 2 hold for any geometry. The shutter position is described by its offset, defined as the distance it has travelled from fully covering Sense 1 (offset 0) toward fully covering Sense 2 (offset 2 m). At $v = 1$ m/s the offset in metres is also the elapsed time in seconds.

4.1. Analytical Result

Fig. 1 shows the analytical solution. The upper panel plots the charge on each sense plate from (4) against shutter offset, and the lower panel plots the amplifier output voltage from (8). The charge on Sense 1 rises linearly from zero to $\epsilon_0 E_0 A = 6.9$ nC as it is uncovered, while the charge on Sense 2 falls by the same amount. Because the charge is linear in offset, the plate current is constant and the output voltage is flat at 3.54 V. The steps at either end of the travel come from the 0.1 m gap between the sense plates. Over the first and last 0.05 m the shutter edge is passing over the gap, so the exposed area of only one plate is changing and the output falls to half.

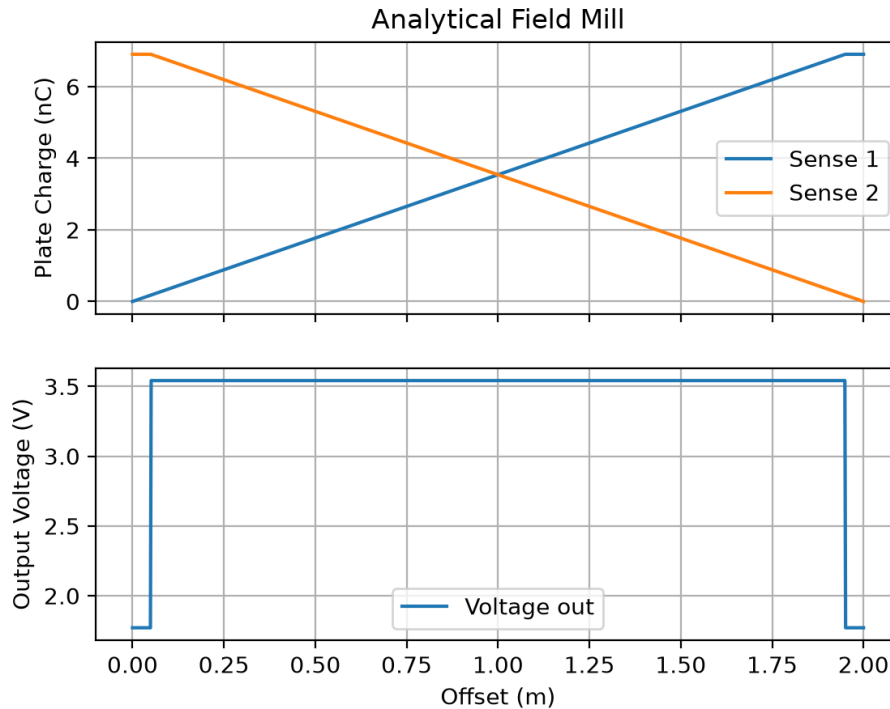


Fig. 1. Analytical solution versus shutter offset. Top: charge on each sense plate. Bottom: transimpedance amplifier output voltage.

4.2. Numerical Field Distribution

Figs. 2–4 show the electric field magnitude $|\mathbf{E}|$ from the numerical solution at three shutter positions: the start of travel, the midpoint, and the end of travel. Colour indicates field magnitude in V/m, the orange rectangles are the sense plates, and the grey rectangle is the grounded shutter. In each map the region beneath the shutter is dark, showing that the plate below it is shielded, while the uncovered plate is exposed to the field arriving from above. The field does not drop

abruptly to zero at the shutter edge but decays gradually into the gap between the shutter and the plates. This fringing is the effect the analytical model neglects. The brightest points lie at the corners of the shutter, where the field concentrates at the sharp conductor edges. The peak value there is set by the grid resolution and should not be read as a physical maximum.

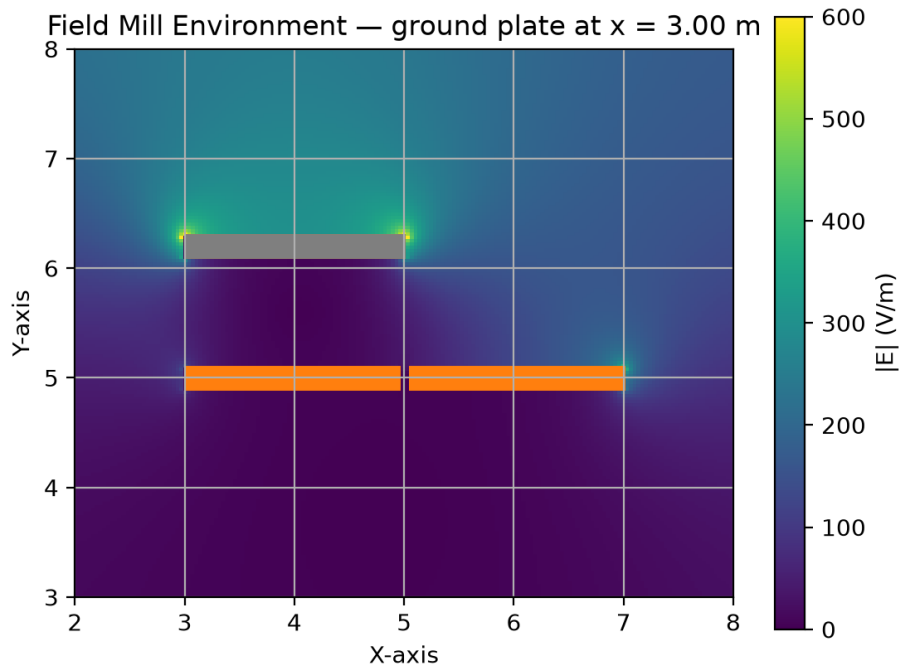


Fig. 2. Electric field magnitude with the shutter (grey) fully covering Sense 1 (offset 0). Sense plates are shown in orange.

4.3. Numerical Plate Charge

Fig. 5 plots the charge on each sense plate, computed from (13), against shutter offset. The charge is negative because the field is directed toward $-y$ and terminates on the plates. As in the analytical model, the two curves mirror each other and cross at mid-travel. Two differences stand out. First, the covered plate is not fully shielded: it retains about a quarter of the charge on the fully exposed plate (-0.16 nC against -0.66 nC), because the field fringes into the gap beneath the shutter. Second, the curves are not straight lines. Each is shallower while its plate is mostly covered and steeper as the plate becomes exposed, so the plate current, and with it the output voltage, varies over the stroke instead of remaining constant.

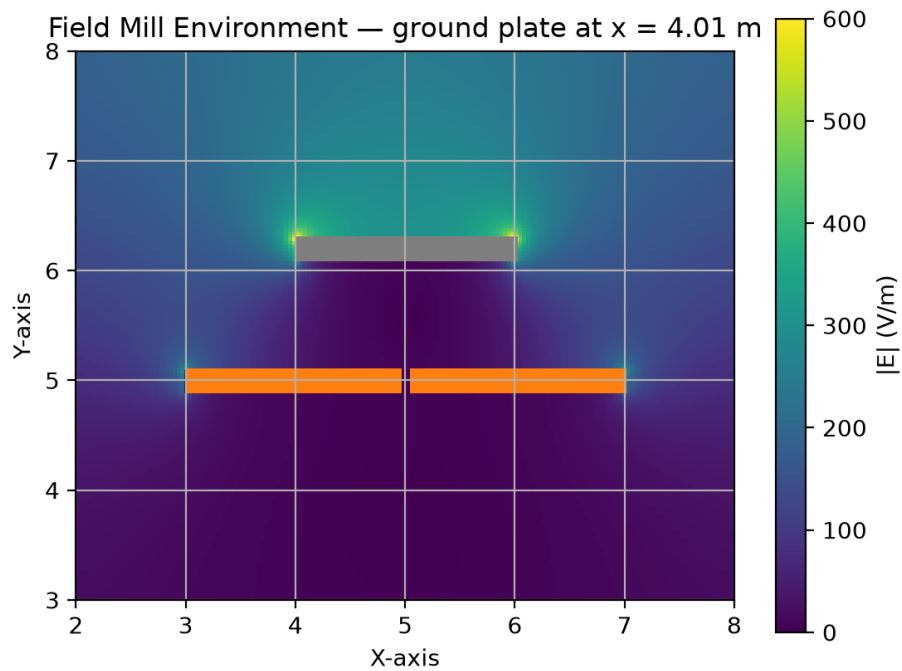


Fig. 3. Electric field magnitude with the shutter at mid-travel (offset 1 m), covering half of each sense plate.

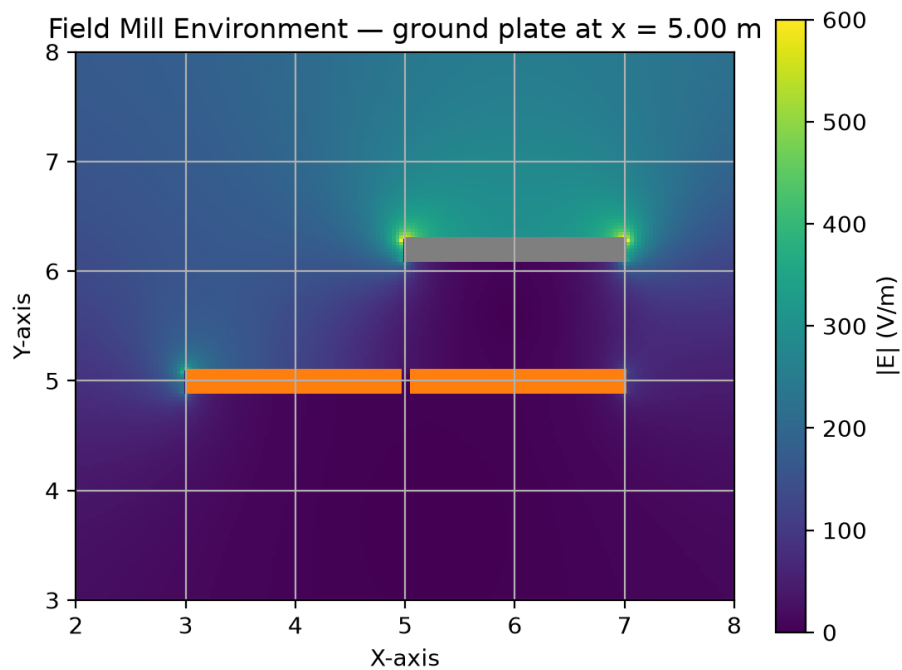


Fig. 4. Electric field magnitude with the shutter fully covering Sense 2 (offset 2 m).

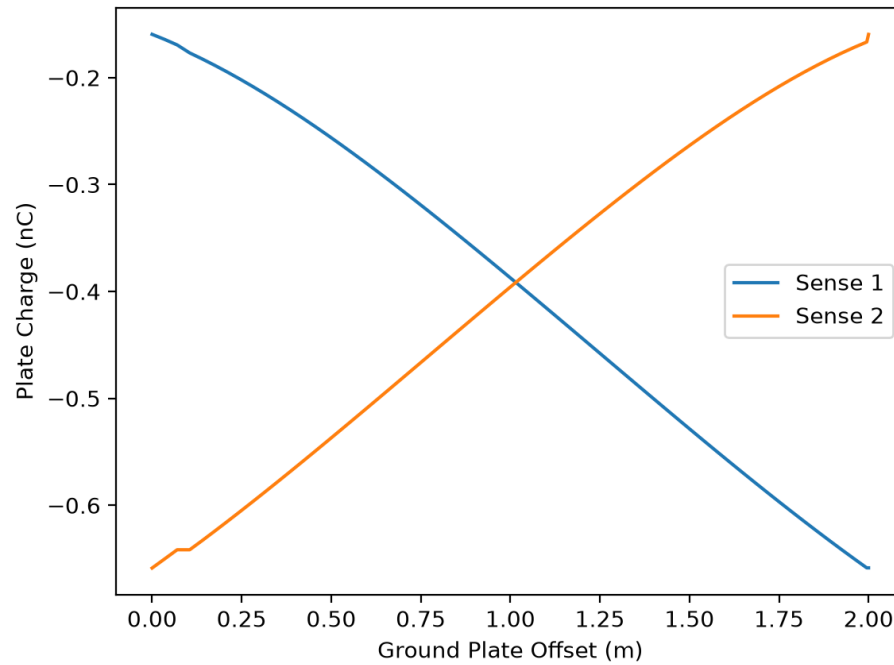


Fig. 5. Numerically computed charge on each sense plate versus shutter offset.

5. Conclusions

The analytical and numerical solutions are identical if not very similar in terms of charge. The only difference being shown in the solutions is the introduction of the fringing elements in the numerical simulation.

6. Appendix 1

```

1 #main
2 #This code is to create an environment to show the use of Gauss' law in a field
  mill.
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from matplotlib.patches import Rectangle
6 #from mpl_toolkits.mplot3d import Axes3D
7 EPS0 = 8.854e-12 # F/m permittivity of free space
8 MU0 = 4*np.pi*1e-7 # H/m permeability of free space
9 C = 1/np.sqrt(MU0 * EPS0) # m/s speed of light
10 # 1. Create Domain space
11 # We can assume that the domain space is 2D if the field mill contains an
  infinite y component.
12 class Domain:
13     def __init__(self, xlim, ylim, dx, dy):
14         self.xlim = xlim
15         self.ylim = ylim
16         self.X, self.Y = np.meshgrid(np.linspace(xlim[0], xlim[1], dx), np.
          linspace(ylim[0], ylim[1], dy))
17         self.Z = np.zeros_like(self.X) # Initialize Z to zero for 2D field
          distribution
18
19 # 2. Create a field mill with a given geometry and parameters
20 # Create a class to represent the field mill, including its geometry and
  parameters.
21 class Plate:
22     def __init__(self, name, x0, y0, width, height, potential=0.0):
23         self.name = name
24         self.x0 = x0 # lower left corner
25         self.y0 = y0
26         self.width = width
27         self.height = height
28         self.potential = potential # V
29
30     def mask(self, X, Y):
31         # True at every grid point that lies inside the plate
32         return ((X >= self.x0) & (X <= self.x0 + self.width) &
33               (Y >= self.y0) & (Y <= self.y0 + self.height))
34
35 class FieldMill:
36     def __init__(self, center=(0,0), side=4, gap=1, thick=0.2,
37                 n_sense=2, sense_space=0.1, shutter=0):
38         self.center = center # center of the sense plate layer
39         self.side = side # m side length of the square mill
40         self.depth = side # m extent into the page (square mill)
41         self.gap = gap # m distance between sense plates and
          ground plate
42         self.thick = thick # m plate thickness in y
43         self.n_sense = n_sense
44         self.sense_space = sense_space # m spacing between neighbouring sense
          plates
45         self.shutter = shutter # 0 = ground plate at the left edge, 1 = at
          the right edge
46
47         self.sense_plates = self._make_sense_plates()
48         self.ground_plate = self._make_ground_plate()
49
50     def _make_sense_plates(self):
51         cx, cy = self.center

```

```

52         width = (self.side - (self.n_sense - 1) * self.sense_space) / self.
           n_sense
53         x_left = cx - self.side / 2
54         y0 = cy - self.thick / 2
55         return [Plate(f"Sense_{i+1}", x_left + i * (width + self.sense_space)
           , y0, width, self.thick)
           for i in range(self.n_sense)]
56
57
58     def _make_ground_plate(self):
59         # The ground plate covers one sense plate at a time and slides in x
           with the shutter position.
60         cx, cy = self.center
61         width = self.side / self.n_sense
62         x0 = cx - self.side / 2 + self.shutter * (self.side - width)
63         y0 = cy + self.thick / 2 + self.gap
64         return Plate("Ground", x0, y0, width, self.thick)
65
66     @property
67     def plates(self):
68         return self.sense_plates + [self.ground_plate]
69
70     def set_shutter(self, shutter):
71         self.shutter = shutter
72         self.ground_plate = self._make_ground_plate()
73
74     def mask(self, X, Y):
75         # True at every grid point that lies inside any conductor
76         mask = np.zeros_like(X, dtype=bool)
77         for plate in self.plates:
78             mask |= plate.mask(X, Y)
79         return mask
80
81     def plot(self, ax):
82         for plate in self.plates:
83             ax.add_patch(Rectangle((plate.x0, plate.y0), plate.width, plate.
               height,
               color="tab:orange", label=plate.name))
84         ground = self.ground_plate
85         ax.add_patch(Rectangle((ground.x0, ground.y0), ground.width, ground.
               height,
               color="tab:gray", label=ground.name))
86
87
88
89 # 3. Define field distribution in the domain space
90 class FieldDistribution:
91     def __init__(self, domain, field_mill, E0=100):
92         self.domain = domain
93         self.field_mill = field_mill
94         self.field = np.zeros_like(domain.X) # Initialize field to zero
95         self.E0 = E0 # V/m example constant field
96         self.dx = domain.X[0,1] - domain.X[0,0]
97         self.dy = domain.Y[1,0] - domain.Y[0,0]
98
99         self.V = np.zeros_like(domain.X) # V
100        self.Ex = np.zeros_like(domain.X) # V/m
101        self.Ey = np.zeros_like(domain.X) # V/m
102        self.E = np.zeros_like(domain.X) # V/m magnitude
103 # 4. Make Gauss' law calculations to determine the field at the field mill
104     def solve(self, V_init, tol=1e-5, max_iter=100000):
105         X,Y = self.domain.X, self.domain.Y
106         dx2,dy2 = self.dx**2, self.dy**2
107
108         if V_init is None:

```

```

109         V = self.E0 * (Y - self.domain.ylim[0])
110     else:
111         V = V_init.copy()
112
113     fixed = np.zeros_like(X, dtype=bool)
114     fixed[0,:] = True
115     fixed[-1,:] = True
116     for plate in self.field_mill.plates:
117         inside = plate.mask(X,Y)
118         V[inside] = plate.potential
119         fixed |= inside
120
121     #Update Equation
122     for i in range(max_iter):
123         Vp = np.pad(V, 1, mode="reflect")
124         Vp_left = Vp[1:-1, :-2]
125         Vp_right = Vp[1:-1, 2:]
126         Vp_up = Vp[:-2, 1:-1]
127         Vp_down = Vp[2:, 1:-1]
128         V_new = ((Vp_left + Vp_right) * dy2)/(2*(dx2 + dy2)) + ((Vp_up +
129             Vp_down) * dx2)/(2*(dx2 + dy2))
130         V_new[fixed] = V[fixed]
131         change = np.max(np.abs(V_new - V))
132         if change < tol:
133             break
134
135     self.V = V
136     self.iterations = i + 1
137     dVdy, dVdx = np.gradient(V, self.dy, self.dx)
138     self.Ex = -dVdx
139     self.Ey = -dVdy
140     self.E = np.hypot(self.Ex, self.Ey)
141     self.E[self.field_mill.mask(X,Y)] = 0
142
143     def plate_charge(self, plate):
144         V = self.V
145         Vp = np.pad(V, 1, mode="reflect")
146         flux = ((2*V - Vp[1:-1, :-2] - Vp[1:-1, 2:]) * self.dy /self.dx +
147             (2*V - Vp[:-2, 1:-1] - Vp[2:, 1:-1]) * self.dx /self.dy)
148         inside = plate.mask(self.domain.X, self.domain.Y)
149         return EPS0 * flux[inside].sum() * self.field_mill.thick
150
151
152     def plot(self, ax):
153         xlim, ylim = self.domain.xlim, self.domain.ylim
154         extent = (xlim[0] - self.dx/2, xlim[1] + self.dx/2,
155             ylim[0] - self.dy/2, ylim[1] + self.dy/2)
156         image = ax.imshow(self.E, origin="lower", extent=extent, cmap="viridis"
157             , vmin=0)
158         self.field_mill.plot(ax)
159         ax.set_xlim(xlim)
160         ax.set_ylim(ylim)
161         return image
162
163     # 5. Calculate the field at the mill and show the plot
164
165     if __name__ == "__main__":
166         # Define the domain space
167         domain = Domain(xlim=(1.5, 8.5), ylim=(2.5, 8.5), dx=201, dy=201)
168         mill = FieldMill(center=(5.0, 5.0))
169         # Create a figure and axis for plotting
170         fig, ax = plt.subplots()

```

```

169 cbar = None
170 V_prev = None
171 charges = {plate.name: [] for plate in mill.sense_plates}
172
173 # Move the ground plate one grid spacing at a time,
174 # from fully covering the left sense plate to fully covering the right
    sense plate
175 step = domain.X[0, 1] - domain.X[0, 0] # m grid spacing in x
176 travel = mill.side - mill.ground_plate.width # m total distance the
    ground plate moves
177 offsets = np.append(np.arange(0, travel, step), travel) # last step lands
    exactly on the right edge
178
179 for offset in offsets:
180     mill.set_shutter(offset / travel)
181     field = FieldDistribution(domain, mill, E0=100)
182     field.solve(V_init=V_prev)
183     V_prev = field.V
184     for plate in mill.sense_plates:
185         charges[plate.name].append(field.plate_charge(plate))
186     # Plot the domain space
187     ax.clear()
188     mill.plot(ax)
189     image = field.plot(ax)
190     if cbar is None:
191         cbar = fig.colorbar(image, ax=ax, label="|E| (V/m)")
192     else:
193         cbar.update_normal(image)
194     ax.set_xlim(2, 8)
195     ax.set_ylim(3, 8)
196     ax.set_title(f"Field_Mill_Environment_ground_plate_at_x={mill.
        ground_plate.x0:.2f}_m")
197     ax.set_xlabel("X-axis")
198     ax.set_ylabel("Y-axis")
199     ax.grid()
200     plt.pause(0.05)
201
202 fig2, ax2 = plt.subplots()
203 for name, q in charges.items():
204     ax2.plot(offsets, np.array(q)*1e9, label=name)
205 ax2.set_xlabel("Ground_Plate_Offset (m)")
206 ax2.set_ylabel("Plate_Charge (nC)")
207 ax2.legend()
208 # Show the plot
209 plt.show()

```